Two-Tower Recommender

An engineering research report on Retrieval + ranking, trained from scratch

Sai Veda

Independent Project Research

2023

Contents

1. Abstract

This project documents a recommendation stack where retrieval and ranking are deliberately separated so candidate generation can scale without giving up downstream business relevance. The combination of two-tower retrieval, ANN candidate generation, and a ranker gives the project a realistic online shape. The measured uplift over popularity and retrieval-only ranking makes the business value concrete. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

Keywords: Retrieval + ranking, trained from scratch, implementation evidence, benchmark results, project validation

2. Introduction

Recommendation quality breaks when systems either over-index on popularity or overfit a ranking model that cannot scale to a full catalog. The architecture therefore mirrors production: first recall, then precision. A complete, offline, deterministic recommender system: a from-scratch two-tower retrieval model, ANN candidate generation, a gradient-boosted ranker, and a leak-free temporal evaluation harness - trained on millions of implicit interactions and architected for 1B. No GPU, no network, no paid APIs.

2.1 Research Problem

Recommendation quality breaks when systems either over-index on popularity or overfit a ranking model that cannot scale to a full catalog. The architecture therefore mirrors production: first recall, then precision.

2.2 Research Questions and Hypotheses

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: beats popularity +76% recall; ranker +7.8% nDCG.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 33 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Technical Background Review

3.1 Design Foundations

1. Retrieval and ranking are separated intentionally because they solve different scale problems. Candidate recall and ranking precision are intentionally split so the serving path scales without giving up business relevance.
2. Temporal evaluation is non-negotiable for recommendation systems that serve future behavior.
3. The item embedding space is treated as infrastructure that downstream ranking can build on.

3.2 Supporting Examples

- Temporal data split and training statistics.
- Two-tower retrieval model and item embedding space.
- ANN candidate recall plus gradient-boosted reranking.

4. Research Method

The project was reviewed as an engineering artefact using repository documentation, architecture notes, benchmark evidence, automated tests, and generated screenshots. Started with a temporal split so evaluation reflects forward-looking recommendation behavior. Trained user and item towers with in-batch negatives and popularity correction. Separated ANN recall from a richer ranking stage that can use additional features. Measured recall lift over popularity and ranking lift over retrieval-only baselines.

5. Data Description

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 33 automated tests, 0 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

6. Analysis

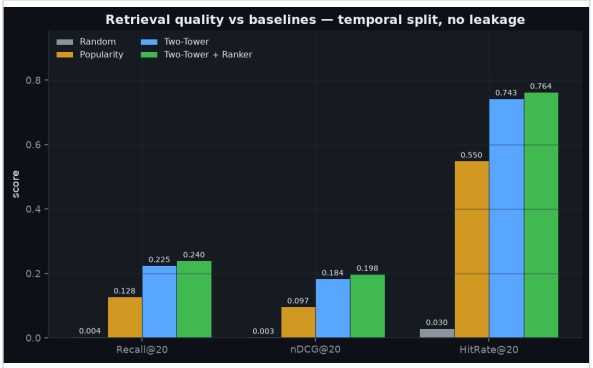
6.1 Benchmark and Validation Results

The headline repository result is reported as: beats popularity +76% recall; ranker +7.8% nDCG. The table below reproduces benchmark evidence included in the repository where available.

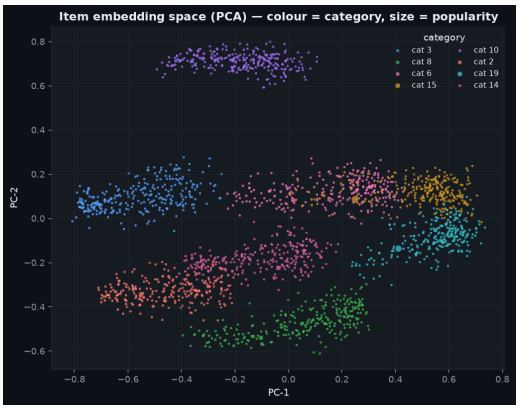
The retrieval gains matter because they increase the candidate pool quality before ranking begins. The ranker lift is easier to explain when shown as an improvement over a strong retrieval baseline. The dashboard evidence makes clear that recall and nDCG improve for different reasons and should be reviewed separately.

6.2 Test Result Screenshots

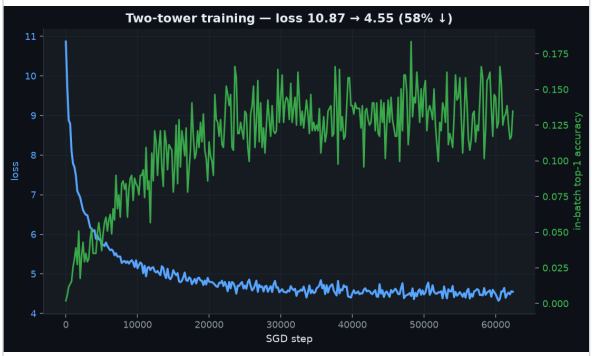
The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.



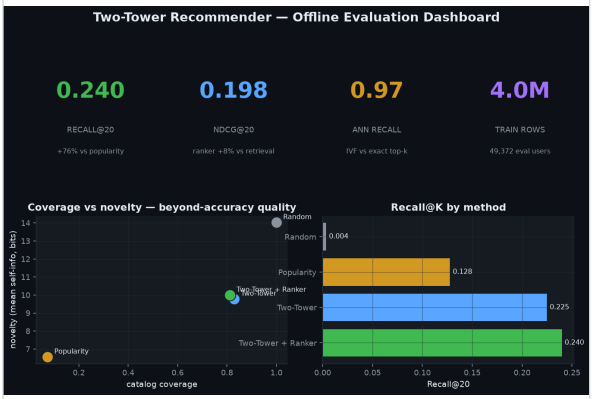
Retrieval quality vs baselines



Item embedding space (PCA), coloured by category



Two-tower training curve



Offline evaluation dashboard

6.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	33 tests committed in repository validation flow.
Benchmark results	Limited	No benchmark markdown tables were present; discussion relies on screenshots and he
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

7. Discussion

The combination of two-tower retrieval, ANN candidate generation, and a ranker gives the project a realistic online shape. The measured uplift over popularity and retrieval-only ranking makes the business value concrete. The retrieval gains matter because they increase the candidate pool quality before ranking begins. The ranker lift is easier to explain when shown as an improvement over a strong retrieval baseline. The dashboard evidence makes clear that recall and nDCG improve for different reasons and should be reviewed separately. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

8. Conclusion

This project document concludes that Two-Tower Recommender is best understood as a complete technical implementation supported by repository-native evidence. This project documents a recommendation stack where retrieval and ranking are deliberately separated so candidate generation can scale without giving up downstream business relevance.

9. Future Work

- Add session-aware or context-aware retrieval features for short-term intent shifts.
- Experiment with harder negative sampling and refreshed item embeddings.
- Introduce business-constraint reranking for diversity, margin, or inventory goals.

10. References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/benchmark_scale.py - benchmark or result artefact used in the analysis section.
4. benchmarks/metrics.csv - benchmark or result artefact used in the analysis section.
5. benchmarks/summary.json - benchmark or result artefact used in the analysis section.
6. tests/ - automated validation suite used to support implementation claims.
7. assets/embedding_space.png - generated screenshot evidence included in the report.
8. assets/kpi_dashboard.png - generated screenshot evidence included in the report.